How Plotly.js renders shapes as SVG DOM elements

Plotly.js **does** add a `data-index` attribute to rendered shape SVG elements, but there is **no official API** to get DOM references for specific shapes. The library uses a layered rendering system that can cause index mismatches between `layout.shapes` array positions and DOM query results when shapes use different layers.

The data-index attribute is present but undocumented

Each shape in `layout.shapes` renders as a `<path>` element within an SVG group, and Plotly.js sets a **`data-index` attribute** corresponding to the shape's position in the `layout.shapes` array. GitHub issue #4931 confirms the rendered HTML structure:

```html
<path data-index="0" fill-rule="evenodd" d="M81 119 L119 119 L119 101 L81 101 Z"
    clip-path="url('#clip157432xy')"
    style="opacity: 1; stroke: rgb(0, 0, 0); fill: red; stroke-width: 2px;">
</path>
```

GitHub

Beyond `data-index`, shapes receive standard SVG styling attributes (`fill`, `stroke`, `stroke-width`, `opacity`, `d` for path data) but **no CSS classes** are applied directly to individual shape elements. The parent container receives the `shapelayer` class. A feature request for an `id` attribute (issue #4931) was closed without implementation. GitHub

DOM structure varies by layer property

The `layer` property fundamentally determines where shapes render in the SVG hierarchy: Plotly

Layer Value	DOM Location	Rendering Position
`"above"` (default)	`g.layer-above > g.shapelayer`	Above all traces
`"below"`	`g.layer-below > g.shapelayer` or `g.layer-subplot > g.shapelayer`	Below gridlines
`"between"`	Inside subplot's layer structure	Between gridlines and traces

Shapes with `layer: 'below'` that reference specific axes go into subplot-specific containers (`g.layer-subplot > g.shapelayer`), while paper-referenced shapes go into the main `g.layer-below` container. `Medium` `Plotly` This architectural decision creates the **index mismatch problem**: a `querySelectorAll('g.shapelayer path')` returns elements in DOM order, not `layout.shapes` array order.

The index mismatch problem explained

When `layout.shapes` contains shapes with mixed layers, DOM traversal order diverges from array order:

```javascript
layout.shapes = [
  { type: 'rect', layer: 'above', ... },   // index 0 → layer-above
  { type: 'rect', layer: 'below', ... },   // index 1 → layer-below
  { type: 'rect', layer: 'above', ... },   // index 2 → layer-above
  { type: 'rect', layer: 'below', ... }    // index 3 → layer-below
];
```

A naive `querySelectorAll('g.shapelayer path')` returns elements in DOM order: **[index 1, index 3, index 0, index 2]** because `layer-below` appears earlier in the SVG tree than `layer-above`. The `data-index` attribute preserves the correct mapping, so **always use `data-index` rather than assuming positional correspondence**.

No official API exists, but workarounds are available

Plotly.js provides no method like `Plotly.getShapeElement()`. The Plotly maintainer etienne explicitly discouraged direct DOM manipulation in 2016, stating developers would need "pretty heavy trickery" to interact with SVG nodes. `plotly` However, several semi-official and community workarounds exist:

Using data-index for reliable selection:

```javascript
// Select specific shape by array index
const shape0 = document.querySelector('path[data-index="0"]');

// Build array matching layout.shapes order
const shapeElements = layout.shapes.map((_, i) =>
  document.querySelector(`path[data-index="${i}"]`)
);
```

Accessing active shape via internal API:

```javascript
// After user clicks an editable shape
const activeIndex = gd._fullLayout._activeShapeIndex;
```

plotly

Event-based DOM manipulation via plotly_afterplot:

```javascript
gd.on('plotly_afterplot', function() {
  const shapePaths = gd.querySelectorAll('g.shapelayer path');
  shapePaths.forEach(path => {
    const index = path.getAttribute('data-index');
    path.setAttribute('data-custom-id', `shape-${index}`);
  });
});
```

Hooking into rendering requires event listeners or source modification

The `plotly_afterplot` event fires after every render, making it the primary hook for post-render DOM manipulation. For shape click events specifically—which Plotly.js does **not** expose—the internal click handler in `draw.js` (line 189) uses:

```javascript
path.node().addEventListener('click', function() { return activateShape(gd, path); });
```

GitHub github

Community member Saratoga documented a source code modification to emit custom events:

```javascript
// Modified draw.js to emit shape selection events
path.node().addEventListener('click', function() {
  const status = activateShape(gd, path);
  if (gd._fullLayout._activeShapeIndex)
    gd.emit('plotly_selected', { activeShapeIndex: gd._fullLayout._activeShapeIndex });
  else
    gd.emit('plotly_deselect', null);
  return status;
});
```

plotly

Without modifying source code, you can intercept shape clicks using native DOM events on the shapelayer container:

```javascript
gd.addEventListener('mousedown', e => {
  if (e.target.closest('.shapelayer')) {
    const path = e.target.closest('path');
    const dataIndex = path?.getAttribute('data-index');
    console.log('Shape clicked:', dataIndex);
  }
});
```

GitHub

No version has added shape identification features

Review of the Plotly.js changelog reveals **no additions** of `id`, `data-*` attributes, or DOM identification features for shapes across any version. Key shape-related additions include:

- **v2.18.2**: Added `label` attribute for text labels on shapes GitHub GitHub
- **v1.56.0**: Added `editable` and `fillrule` attributes; introduced draw modes GitHub GitHub
- **v2.24/v2.33**: Added `layer: 'between'` option (PR #6927) GitHub GitHub
- **v1.37.0**: Added `xsizemode`, `ysizemode` for fixed-size shapes

Feature request #4931 for an `id` attribute was closed without implementation. The most-upvoted shape feature request (#4780) with **133 upvotes** asks for hover and text on shapes—it remains in the P3 backlog after years. GitHub github

Reliable selector patterns for different scenarios

Purpose	Selector
All shapes above traces	`g.layer-above > g.shapelayer path`
All shapes below traces	`g.layer-below g.shapelayer path, g.layer-subplot g.shapelayer path`
Specific shape by index	`path[data-index="N"]`
All shapes regardless of layer	`g.shapelayer path`

Conclusion

The **`data-index` attribute is the only reliable identifier** Plotly.js provides for mapping rendered SVG elements to `layout.shapes` array positions. While undocumented, this attribute has remained stable across versions and correctly handles the layer-induced DOM ordering differences. For applications requiring shape-specific DOM manipulation, the recommended approach combines `plotly_afterplot` event listening with `data-index`-based selection, avoiding assumptions about DOM traversal order. Native shape click events and official DOM reference APIs remain unimplemented feature requests in Plotly's backlog.